RCS Platform API Consumer Guide

Version 1 | Production base URL https://rcs.ostpl.com

This guide explains how an authenticated RCS Platform user can create templates, create campaigns, monitor processing, and download campaign reports. All protected requests use a Bearer access token generated from the same username and password used in the portal.

Audience Application developers and integration teams

API prefix /api/v1

Data format JSON unless the endpoint specifies multipart file upload or CSV download

Quick start

Complete these steps in order for the first integration test.

1. Generate an access token with your portal username and password.
2. List approved templates and select an approved template ID.
3. Create a campaign. The server generates an idempotency key automatically.
4. Poll the campaign status endpoint until processing completes.
5. Download the report as a CSV file.

Endpoint summary

Method	Endpoint	Purpose
POST	/api/v1/auth/token	Generate access and refresh tokens
POST	/api/v1/auth/refresh	Refresh an expired access token
GET	/api/v1/auth/me	Read the authenticated user
POST	/api/v1/templates	Create a template
GET	/api/v1/templates	List owned templates
GET	/api/v1/templates/approved	List approved templates
GET	/api/v1/templates/{template_id}	Read template details
GET	/api/v1/templates/{template_id}/status	Read approval status
POST	/api/v1/campaigns	Create a campaign from JSON
POST	/api/v1/campaigns/upload	Create a campaign from CSV or TXT
GET	/api/v1/campaigns	List owned campaigns
GET	/api/v1/campaigns/{campaign_id}	Read campaign details
GET	/api/v1/campaigns/{campaign_id}/status	Monitor campaign totals
GET	/api/v1/campaigns/{campaign_id}/report	Download a CSV report

Authentication

Access tokens authorize API calls. Refresh tokens obtain a new token pair without resending the password. Store both tokens in a secure server-side secret store and never place them in URLs or application logs.

POST `/api/v1/auth/token`

Generate an access token and refresh token.

Authentication Username and password in the JSON body

```
curl -X POST "https://rcs.ostpl.com/api/v1/auth/token" \
  -H "Content-Type: application/json" \
  -d '{"username":"YOUR_USERNAME","password":"YOUR_PASSWORD"}'
```

Successful response

```
{
  "access_token": "eyJ...",
  "refresh_token": "eyJ...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "user": {
    "id": "USER_ID",
    "username": "YOUR_USERNAME",
    "name": "Account Name",
    "role": "user"
  }
}
```

POST `/api/v1/auth/refresh`

Generate a new access and refresh token pair.

Authentication Refresh token in the JSON body

```
curl -X POST "https://rcs.ostpl.com/api/v1/auth/refresh" \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"REFRESH_TOKEN"}'
```

GET `/api/v1/auth/me`

Confirm the access token and return the active user.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/auth/me" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Bearer header

Include this header on every protected request:

```
Authorization: Bearer ACCESS_TOKEN
```

Template APIs

Administrators create approved templates immediately. Templates created by users or resellers enter pending status and must be approved before campaign creation.

POST `/api/v1/templates`

Create a plain, text, rich, or carousel template.

Authentication Bearer token required

```
curl -X POST "https://rcs.ostpl.com/api/v1/templates" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "October Offer",
    "template_type": "plain",
    "content": {"text": "Your offer message"}
  }'
```

Successful response

```
{
  "status": "success",
  "template_id": "TEMPLATE_ID",
  "approval_status": "pending",
  "message": "Template submitted for admin approval"
}
```

GET `/api/v1/templates`

List templates owned by the authenticated user. Optional query parameters are status, limit, and offset.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/templates?status=approved&limit=50&offset=0" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

GET `/api/v1/templates/approved`

List templates that can be used to create a campaign.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/templates/approved" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

GET `/api/v1/templates/{template_id}`

Read template metadata and content.

Authentication Bearer token required

GET `/api/v1/templates/{template_id}/status`

Read pending, approved, or rejected status and the administrator comment.

Authentication Bearer token required

Rich and carousel content

Rich and carousel templates must contain the Jio RCS structure under content.data.content.richCardDetails. Use the portal template builder or an already approved payload as the structural reference.

Campaign APIs

Campaign creation checks the approved template, account balance, provider configuration, unique valid mobile numbers, and wallet debit before placing upload work in the existing background queue.

Optional Idempotency Key

The Idempotency-Key header is optional. When it is omitted, the server generates a UUID automatically and returns it as idempotency_key in the campaign response. Integrations may still supply an 8 to 128 character key when they need to retry the same request safely. Retrying an identical completed request with the same supplied key returns the original response. Reusing that key with different input returns HTTP 409.

```
Idempotency-Key: order-20261007-0001
```

POST `/api/v1/campaigns`

Create a campaign using a JSON number array. Use this endpoint for no more than 100,000 submitted values.

Authentication Bearer token required

```
curl -X POST "https://rcs.ostpl.com/api/v1/campaigns" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "campaign_name": "October Campaign",
    "template_id": "TEMPLATE_ID",
    "numbers": ["9876543210", "9123456789"]
  }'
```

Accepted response

```
{
  "status": "success",
  "message": "Campaign accepted. Processing in background.",
  "campaign_id": "CAMPAIGN_ID",
  "idempotency_key": "AUTO_GENERATED_UUID"
}
```

POST `/api/v1/campaigns/upload`

Create a campaign from a UTF-8 CSV or TXT file. Use this endpoint for large campaigns.

Authentication Bearer token required

```
curl -X POST "https://rcs.ostpl.com/api/v1/campaigns/upload" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -F "campaign_name=October Campaign" \
  -F "template_id=TEMPLATE_ID" \
  -F "numbers_file=@numbers.csv"
```

Upload rule	Limit
Encoding	UTF-8 or UTF-8 with BOM
Supported format	CSV or plain text
Maximum file size	20 MB
Maximum unique valid numbers	500,000
Accepted mobile format	10 digit Indian mobile starting with 6, 7, 8, or 9; optional 91 prefix

Campaign monitoring

GET `/api/v1/campaigns`

List campaigns owned by the authenticated user. Optional query parameters are status, limit, and offset.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/campaigns?limit=50&offset=0" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

GET `/api/v1/campaigns/{campaign_id}`

Read campaign metadata and delivery totals.

Authentication Bearer token required

GET `/api/v1/campaigns/{campaign_id}/status`

Poll campaign status and counters while processing continues.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/campaigns/CAMPAIGN_ID/status" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Representative response

```
{
  "id": "CAMPAIGN_ID",
  "campaign_name": "October Campaign",
  "template_id": "TEMPLATE_ID",
  "template_name": "October Offer",
  "status": "completed",
  "total_numbers": 100000,
  "queued_count": 0,
  "sent_count": 0,
  "delivered_count": 69000,
  "read_count": 0,
  "clicked_count": 0,
  "replied_count": 0,
  "failed_count": 31000
}
```

Campaign report download

GET `/api/v1/campaigns/{campaign_id}/report`

Stream an authorized campaign report as CSV. Use the optional status query parameter to download one status only.

Authentication Bearer token required

```
curl "https://rcs.ostpl.com/api/v1/campaigns/CAMPAIGN_ID/report?status=delivered" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  --output campaign-report.csv
```

The CSV contains these columns: Mobile, Status, Template Name, and Sent Time (IST).

Python example

```
import requests

base_url = "https://rcs.ostpl.com"

token_response = requests.post(
    f"{base_url}/api/v1/auth/token",
    json={"username": "YOUR_USERNAME", "password": "YOUR_PASSWORD"},
    timeout=30,
)
token_response.raise_for_status()
tokens = token_response.json()
headers = {"Authorization": f"Bearer {tokens['access_token']}"}

templates = requests.get(
    f"{base_url}/api/v1/templates/approved",
    headers=headers,
    timeout=30,
).json()

campaign = requests.post(
    f"{base_url}/api/v1/campaigns",
    headers=headers,
    json={
        "campaign_name": "API Campaign",
        "template_id": templates["items"][0]["id"],
        "numbers": ["9876543210", "9123456789"],
    },
    timeout=60,
)
campaign.raise_for_status()
campaign_id = campaign.json()["campaign_id"]

with requests.get(
    f"{base_url}/api/v1/campaigns/{campaign_id}/report",
    headers=headers,
    stream=True,
    timeout=300,
) as report:
    report.raise_for_status()
    with open("campaign-report.csv", "wb") as output:
        for chunk in report.iter_content(256 * 1024):
            output.write(chunk)
```

JavaScript example

```
const baseUrl = "https://rcs.ostpl.com";

const tokenResponse = await fetch(`${baseUrl}/api/v1/auth/token`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    username: "YOUR_USERNAME",
    password: "YOUR_PASSWORD"
  })
});

if (!tokenResponse.ok) throw new Error(await tokenResponse.text());
const tokens = await tokenResponse.json();

const campaignResponse = await fetch(`${baseUrl}/api/v1/campaigns`, {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${tokens.access_token}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    campaign_name: "API Campaign",
    template_id: "TEMPLATE_ID",
    numbers: ["9876543210", "9123456789"]
  })
});

if (!campaignResponse.ok) throw new Error(await campaignResponse.text());
console.log(await campaignResponse.json());
```

Errors and retry behavior

HTTP status	Meaning	Client action
400	Invalid request, ID, template, file, or mobile input	Correct the request before retrying
401	Missing, invalid, or expired token	Refresh the token or authenticate again
402	Insufficient wallet balance	Add balance before creating the campaign
403	The authenticated user cannot access the resource	Use the correct account; do not retry unchanged
404	Template or campaign not found	Verify the resource ID and ownership
409	Idempotency request conflict or request still processing	Reuse the key only for the identical completed request
413	JSON list, upload file, or unique number count is too large	Use file upload or reduce the request size
422	Required field or data type is missing	Correct the body, form fields, or headers
500	Unexpected server error	Record the request ID and contact support before retrying

Recommended retry rules

- Do not retry HTTP 400, 402, 403, 404, 413, or 422 without changing the request or account state.
- For temporary HTTP 500 or network failures, retry with exponential backoff. If the client supplied an Idempotency-Key, reuse that same key.
- When a client supplies an Idempotency-Key, do not generate a new one merely because the original campaign response was not received.
- Refresh an expired access token once, then repeat the protected request.

Security requirements

- Use HTTPS only.
- Keep passwords, access tokens, and refresh tokens out of URLs and logs.
- Store tokens in a server-side secret store. Do not embed production tokens in browser source code.
- Use a separate platform user for each integration where practical.
- Users can access only their own templates and campaigns. Administrators may access any campaign by ID.
- Remove stored tokens when the integration is disabled or the account is deactivated.

Portal and interactive reference

Signed-in users can open the API page from the portal sidebar. The page contains the authentication flow, endpoint list, this document download, and a link to the interactive OpenAPI reference at https://rcs.ostpl.com/docs.